

```
import numpy as np ,pandas as pd,matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
```

```
x,y_true=make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=1.2,
    random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
x_tr,x_te,y_tr,y_te=train_test_split(
    x,y_true,
    test_size=0.2,
    random_state=42
)
clf=LogisticRegression()
clf.fit(x_tr,y_tr)
print(f'supervised accuracy:{ clf.score(x_te,y_te):.2%}')
```

supervised accuracy:100.00%

```
from sklearn.cluster import KMeans
kmeans=KMeans(n_clusters=3,random_state=42,n_init=10)
cluster_labels=kmeans.fit_predict(x)
print('cluster assignments(first 10):',cluster_labels[:10])
```

cluster assignments(first 10): [1 1 0 2 1 2 0 2 0 0]

```
def manual_kmeans(x, k=3, max_iter=10,seed=42):
    rng = np.random.default_rng(seed)
    centres=x[rng.choice(len(x),k,replace=False)]
    for iteration in range(max_iter):
        dists=np.linalg.norm(x[:,np.newaxis]-centres,axis=2)
        labels=np.argmin(dists,axis=1)
        new_centres=np.array([x[labels==k_].mean(axis=0) for k_ in range(k)])
        print(f'Iter {iteration+1}: centre moved by {np.linalg.norm(new_centres-centres):.4f}')
        if np.allclose(centres,new_centres):print('->converged!'); break
        centres=new_centres
    return labels, centres
x,_=make_blobs(n_samples=200,centers=3,random_state=42)
labels,centres=manual_kmeans(x,k=3)
```

Iter 1: centre moved by 6.2785
 Iter 2: centre moved by 3.6678
 Iter 3: centre moved by 0.0000
 ->converged!

```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
km = KMeans(
    n_clusters=3,
    init='k-means++', # Smart initialisation (better than random)
    n_init=10, # Run 10 times with different starts, keep best
    max_iter=300, # Max iterations per run
    random_state=42
)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_3743/2332344384.py in <cell line: 0>()
----> 1 scaler = StandardScaler()
      2 x_scaled = scaler.fit_transform(x)
      3 km = KMeans(
      4 n_clusters=3,
      5 init='k-means++', # Smart initialisation (better than random)

NameError: name 'StandardScaler' is not defined
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.